Programming for AI

Week 5 Activity

Task 1: The "Currency"

You are building a core module for a banking app. Instead of using raw floats (which are dangerous for money), you need a Currency class that handles its own addition and formatting.

Requirements:

1. **__init__**: Store amount (float) and code (string, e.g., "PKR").
2. **__add__**: Allow adding two Currency objects. **Constraint:** Raise a ValueError if the currency codes do not match (e.g., adding USD to PKR).
3. **__str__**: Return a formatted string like PKR 5,000.00.
4. **__repr__**: Return a developer-friendly string: Currency(amount=5000, code='PKR').

Sample Call:

```
c1 = Currency(500, "USD")
c2 = Currency(300, "USD")
print(c1 + c2)
```

Task 2: The "Smart Building" Hierarchy

A Smart City controller manages devices that are both PowerConsuming and NetworkEnabled.

Requirements:

1. **Component**: Base class with a startup() method.
2. **PowerDevice & NetworkDevice**: Both inherit from Component. Both should override startup(), print their specific initialization (e.g., "Power Grid Connected"), and call super().startup().
3. **SmartLift**: Inherits from both.

Sample Call:

```
lift = SmartLift()
lift.startup()
print(f"MRO Order: {[c.__name__ for c in SmartLift.mro()]}")
```

Task 3: The "E-Commerce" Plugin

You have various Product classes. Some need to be "Shippable" (have weight/dimensions), and some need to be "Discountable" (have seasonal sales logic). You should add the modular features without deep inheritance tree.

Requirements:

1. **ShippingMixin**: Adds weight attribute and a calculate_freight() method. Freight is $10 per kg increments with minimum value of $10.
2. **DiscountMixin**: Adds a apply_discount(percent) method that modifies the price.
3. **Laptop**: A class that inherits from Product, ShippingMixin, and DiscountMixin.

Sample Call:

```
macbook = Laptop(name="Macbook Air", price=1200, weight= 1.2)
macbook.apply_discount(10)
```

Task 4: The "Payment Gateway" Interface

Your app supports multiple payment methods (Credit Card, PayPal, Crypto). You must ensure every payment class has exactly the same methods so the checkout logic doesn't break. The system should enforce the contract for external integrations.

Requirements:

1. **PaymentProvider (ABC)**: Define @abstractmethod for authorize() and settle().
2. **CreditCardPay**: Another abstract class derived from Payment Provider which tracks fees. Also tprovides card number (20 digits), card holder's name, expiry date information. When displayed only shows the last 4 digits of the card.

3. **StripePay**: Must implement both methods. Uses API key to manage transactions. Applies 2.5% surcharge / fee on payment amount at authorization.
4. **VisaPay**: Derived from CreditCardPay. Must implement both methods. Applies 2.5% or 10.0 rupee surcharge / fee which ever is lowest on the payment amount at settlement.
5. **UnionPay**: Derived from CreditCardPay. Must implement both methods. Applies flat 8.0 rupee surcharge / fee at settlement.
6. **RaastPay**: Derived from CreditCardPay. No surcharge / fee is applied.
7. **Validation**: Try to create a GenericPayment class that inherits from PaymentProvider but forgets to implement settle(). Verify it cannot be instantiated.

Sample Call:

```
pay = StripePay(api_key="sk_test_123")
pay.authorize(50.0)
pay.settle()
```

Task 5: The "Smart Signal"

Create a class that behaves like a mathematical vector using Python protocols.

Requirements:

1. Create a class Signal that takes a list of numeric values.
2. Dunders to Implement:
   - __len__: Returns the number of samples in the signal.
   - __add__: Allows adding two Signal objects together (element-wise addition).
   - __getitem__: Allows accessing a sample by index (e.g., sig[5]).
   - __str__: Returns a formatted string: Signal(Size: 10, Mean: 0.85).

Sample Call:

```
sig_a = Signal([1.0, 2.0, 3.0, 4.0])
sig_b = Signal([0.5, 1.5, 2.5, 3.5])

print(f"Length of sig_a: {len(sig_a)}")
print(f"Sample at index 2: {sig_a[2]}")

sig_c = sig_a + sig_b
print(f"Result of addition: {sig_c.values}")
print(sig_c)
```

Task 6: The "Safe Class" Factory

In a high-security financial system, every "Transaction" class **must** be immutable (read-only) after it is created. You should intercept the class creation to enforce structural rules.

Requirements:

1. **ImmutableMeta**: A metaclass that overrides __new__.
2. **Logic**: Inside __new__, iterate through the class dictionary. If any method starts with set_, raise a *TypeError* to prevent developers from adding setter methods to sensitive transaction classes.
3. **WireTransfer**: A class using ImmutableMeta.

Sample Call:

```
try:
      class WireTransfer(metaclass=ImmutableMeta):
      def set_amount(self, val):    # This should trigger the Metaclass error
      self.amount = val
except TypeError as e:
      print(f"Metaclass blocked class creation: {e}")
```